

4-Cell Battery Management and Safety Monitoring System

1. Abstract

The **4-Cell Intelligent Battery Management and Safety Monitoring System** is an embedded battery intelligence and protection platform designed to monitor, analyze, protect, diagnose, and remotely visualize a simulated four-cell lithium battery pack in real time. The system integrates an ESP32-based embedded controller with analog cell-voltage sensing, event-driven safety logic, local human-machine interaction, fault-tolerant runtime management, Blynk cloud telemetry, and an executive-level monitoring dashboard.

The system continuously acquires individual cell voltages and processes the measurements using embedded signal-processing techniques to calculate cell voltage, pack voltage, average voltage, voltage difference, imbalance percentage, and the weakest and strongest cells. Based on these parameters, the battery is classified into health states including **Healthy, Minor Imbalance, Critical Imbalance, and Pack Failure**.

A non-blocking, event-driven protection kernel monitors safety-critical conditions including weak-cell voltage, overvoltage, sensor anomalies, imbalance, and rapid voltage fluctuations. The protection controller uses `millis()`-based scheduling instead of blocking delays and manages relay cut-off, buzzer alerts, LED indications, LCD warnings, recovery confirmation, and anti-relay-chatter mechanisms.

The system also provides an intelligent embedded HMI through a 20×4 I2C LCD. Multiple diagnostic screens automatically rotate to display live cell measurements, pack analytics, health information, protection status, and fault information. During critical conditions, fault-related information receives higher display priority.

A fault-tolerant runtime architecture is used to detect abnormal sensor conditions and manage system operating states. Cloud connectivity is implemented using the Blynk IoT platform, enabling remote visualization of battery status and operational information. The final dashboard provides structured visualization of cell voltages, pack-level analytics, protection state, relay and buzzer status, fault information, and voltage trends.

The complete system is developed as an integrated Wokwi simulation using an ESP32, simulated four-cell voltage inputs, LCD, relay, buzzer, LEDs, and Blynk cloud connectivity. The architecture demonstrates how embedded battery monitoring, safety protection, diagnostics, fault handling, and cloud-based monitoring can be integrated into a scalable battery management platform.

2. Introduction

A multi-cell battery pack requires individual cell monitoring because different cells can develop different voltage and operating conditions.

The proposed system monitors all four cells and provides:

- Cell-voltage monitoring
- Battery analytics
- Imbalance detection
- Fault detection
- Safety protection
- Local LCD monitoring
- Cloud monitoring

The system uses a **non-blocking millis()-based architecture** so that monitoring, protection, display, and communication can operate together.

3. Problem Statement

The system must continuously monitor a simulated four-cell battery pack and identify:

- Individual cell voltage
- Pack voltage
- Average voltage
- Weakest cell
- Strongest cell
- Voltage imbalance
- Battery health
- Fault conditions

When an unsafe condition occurs, the system must provide an appropriate protection response.

4. Project Objectives

The main objectives are:

1. Monitor four individual battery cells.

2. Calculate important battery parameters.
 3. Detect cell imbalance and abnormal conditions.
 4. Classify battery health.
 5. Implement event-driven safety protection.
 6. Control relay, buzzer, LEDs, and LCD.
 7. Provide fault detection and recovery.
 8. Monitor the system through Blynk.
 9. Test the complete system using Wokwi.
-

5. System Scope

The project covers:

- Battery monitoring
- Battery analytics
- Fault detection
- Safety protection
- LCD HMI
- Blynk cloud monitoring
- Wokwi simulation

Four potentiometers are used to represent the four battery cells.

Cell ESP32 Pin

Cell 1 GPIO32

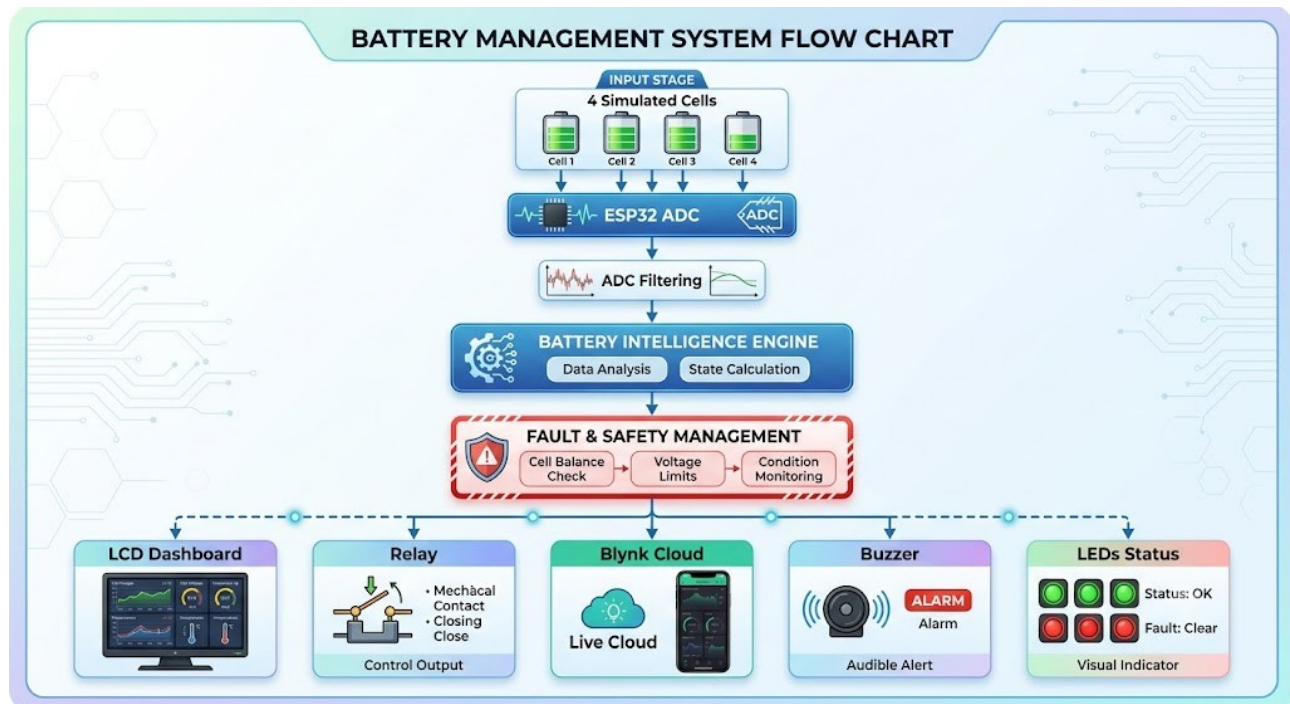
Cell 2 GPIO33

Cell 3 GPIO34

Cell 4 GPIO35

The simulated cell-voltage range is **2.50 V–4.20 V**.

6. System Architecture



The main processing blocks are **sensing** → **filtering** → **battery analysis** → **fault detection** → **protection** → **monitoring**.

7. Battery Intelligence Engine

The Battery Intelligence Engine processes the four cell measurements.

Main calculations

Pack Voltage

$$V_{\text{pack}} = V_1 + V_2 + V_3 + V_4$$

Average Voltage

$$V_{\text{avg}} = (V_1 + V_2 + V_3 + V_4) / 4$$

Voltage Difference

$$V_{\text{diff}} = V_{\text{max}} - V_{\text{min}}$$

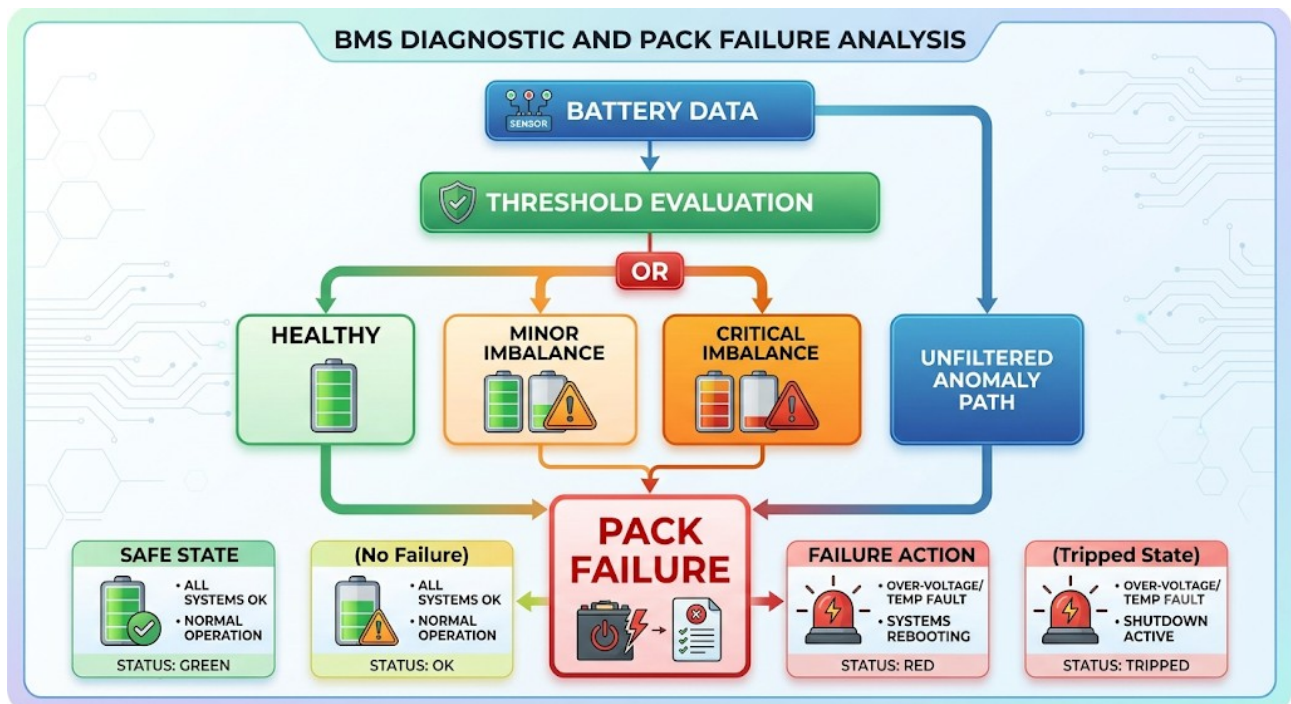
Imbalance

$$\text{Imbalance (\%)} = (V_{\text{max}} - V_{\text{min}}) / V_{\text{avg}} \times 100$$

The system also identifies the **weakest and strongest cells**.

ADC readings are stabilized using averaging and EMA filtering.

The system classifies the battery into:



8. Safety Protection

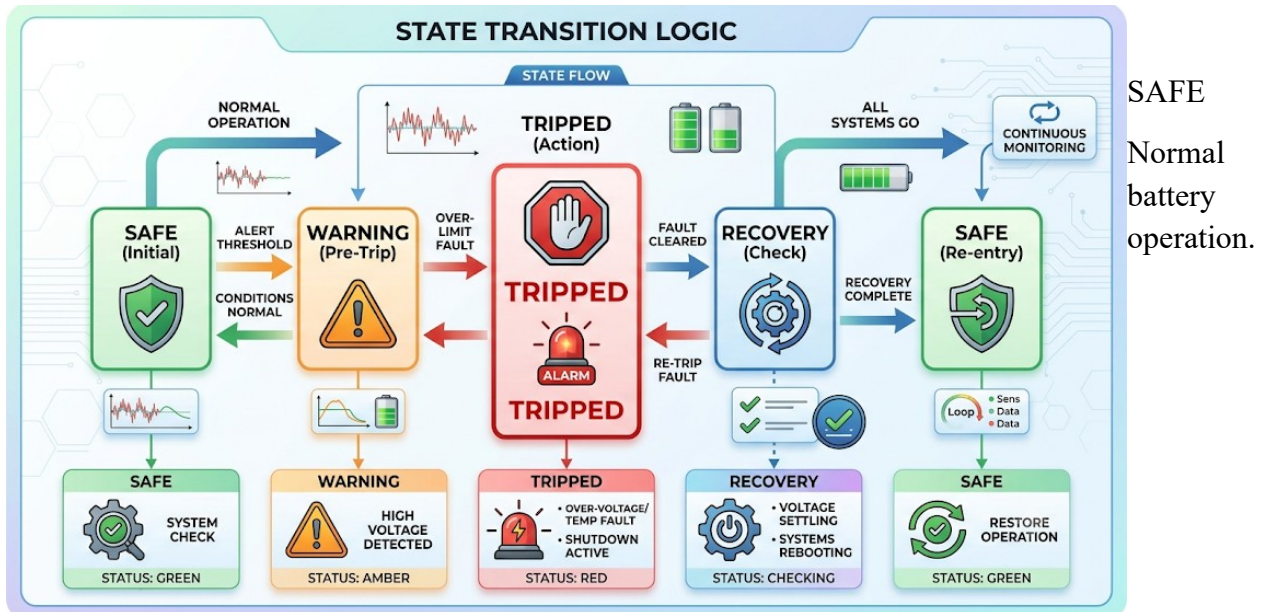
The system detects six major fault conditions:

1. Undervoltage
2. Overvoltage
3. Pack abnormality
4. Excessive imbalance
5. Rapid voltage fluctuation
6. Invalid sensor data

Protection is event-driven and uses timing mechanisms to avoid false trips and relay chattering.

9. Finite State Machine Design

The system uses four states:



WARNING

Abnormal condition detected but protection has not yet tripped.

TRIPPED

Confirmed unsafe condition. Relay protection, buzzer, and red LED are activated.

RECOVERY

Fault has cleared and the system checks that conditions remain safe before returning to SAFE.

State Transition Table

Current State	Condition	Next State
SAFE	Normal	SAFE
SAFE	Warning condition	WARNING
WARNING	Condition cleared	SAFE
WARNING	Confirmed unsafe condition	TRIPPED
TRIPPED	Unsafe condition remains	TRIPPED
TRIPPED	Safe + minimum OFF time	RECOVERY
RECOVERY	Safe condition stable	SAFE
RECOVERY	Fault returns	TRIPPED

10. HMI and Outputs

The system uses a **20×4 I2C LCD** for local monitoring.

The LCD displays:

- Cell voltages
- Pack information
- Battery health
- Protection status
- Diagnostics
- Fault information

Outputs`

Output	Purpose
Relay	Protection/cutoff
Buzzer	Fault warning
Green LED	Normal
Yellow LED	Warning
Red LED	Critical fault

The LCD automatically rotates through information screens and gives priority to critical faults.

Code:

```
#define BLYNK_PRINT Serial
#define BLYNK_TEMPLATE_ID "TMPL3N1_Y48NQ"
#define BLYNK_TEMPLATE_NAME "battery management"
#define BLYNK_AUTH_TOKEN "vzeKgBdL4Ub8Ucuhee9CqW9C0HRRM0q-" // <-- paste your
real device token here

#include <Arduino.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <math.h>

// =====
// WIFI (Wokwi's built-in simulated network)
// =====
char ssid[] = "Wokwi-GUEST";
char pass[] = "";
BlynkTimer blynkTimer;
```

```

// =====
// BLYNK VIRTUAL PIN MAP (matches your datastream table exactly)
// =====
#define BLYNK_RELAY          V0
#define BLYNK_CELL1          V1
#define BLYNK_CELL2          V2
#define BLYNK_CELL3          V3
#define BLYNK_CELL4          V4
#define BLYNK_BUZZER          V5
#define BLYNK_PACK_VOLTAGE    V6
#define BLYNK_AVERAGE_VOLTAGE V7
#define BLYNK_IMBALANCE       V8
#define BLYNK_WEAKEST_CELL    V9
#define BLYNK_STRONGEST_CELL  V10
#define BLYNK_SYSTEM_STATUS   V11
#define BLYNK_HEALTH           V12
#define BLYNK_PROTECTION       V13

// =====
// HARDWARE PINS
// =====
static const uint8_t CELL_COUNT = 4;
static const uint8_t CELL_ADC_PIN[CELL_COUNT] = {32, 33, 34, 35};

#define RELAY_PIN      25
#define BUZZER_PIN     26
#define LED_GREEN_PIN  27
#define LED_YELLOW_PIN 14
#define LED_RED_PIN    12

#define LCD_SDA_PIN  21
#define LCD_SCL_PIN  22
#define LCD_I2C_ADDR 0x27
#define LCD_COLS     20
#define LCD_ROWS      4

LiquidCrystal_I2C lcd(LCD_I2C_ADDR, LCD_COLS, LCD_ROWS);
// =====
// ADC / SIMULATION CONSTANTS
// =====
#define ADC_MAX_VALUE    4095.0f
#define SIM_CELL_MIN_V   2.50f
#define SIM_CELL_MAX_V   4.20f
#define ADC_SAMPLE_COUNT 8
#define FILTER_ALPHA      0.30f

// =====
// HEALTH / PROTECTION THRESHOLDS
// =====
#define MIN_SAFE_VOLTAGE 2.80f

```

```

#define MAX_SAFE_VOLTAGE      4.25f
#define MINOR_IMBALANCE_PCT   2.0f
#define CRITICAL_IMBALANCE_PCT 5.0f

#define UNDERVOLTAGE_V        2.90f
#define OVERVOLTAGE_V          4.22f
#define RECOVERY_LOW_V         3.00f
#define RECOVERY_HIGH_V        4.18f
#define IMBALANCE_TRIP_PCT     6.0f

#define RAPID_DELTA_V          0.35f
#define RAPID_MIN_DELTA_V      0.05f
#define RAPID_CONFIRM_COUNT    3

// =====
// TIMING (all millis()-based - no delay() anywhere below)
// =====
#define ADC_INTERVAL_MS        100UL
#define ANALYTICS_INTERVAL_MS  100UL
#define PROTECTION_INTERVAL_MS 100UL
#define LCD_INTERVAL_MS        500UL
#define LCD_ROTATE_MS          3000UL
#define SERIAL_INTERVAL_MS     2000UL
#define FAULT_CONFIRM_MS        800UL
#define RELAY_MIN_OFF_MS        3000UL
#define RECOVERY_CONFIRM_MS     5000UL
#define BLYNK_HEARTBEAT_MS     10000UL

#define SENSOR_INVALID_LOW     50
#define SENSOR_INVALID_HIGH    4045

// =====
// ENUMS
// =====
enum BatteryHealth { HEALTHY, MINOR_IMBALANCE, CRITICAL_IMBALANCE, PACK_FAILURE };
enum ProtectionState { PROTECTION_SAFE, PROTECTION_WARNING, PROTECTION_TRIPPED,
PROTECTION_RECOVERY };
enum ProtectionReason { REASON_NONE, REASON_UNDERVOLTAGE, REASON_OVERVOLTAGE,
REASON_IMBALANCE, REASON_RAPID_CHANGE, REASON_INVALID_SENSOR };

// =====
// STATE STRUCTS
// =====
struct BatteryState {
    int    rawADC[CELL_COUNT];
    float  cellVoltage[CELL_COUNT];
    float  previousCellVoltage[CELL_COUNT];
    float  packVoltage, averageVoltage;
    float  highestVoltage, lowestVoltage;
    float  voltageDifference, imbalancePercent;
    int    strongestCellIndex, weakestCellIndex;

```

```

    BatteryHealth health;
    bool sensorValid[CELL_COUNT];
};

struct ProtectionKernel {
    ProtectionState state;
    ProtectionReason reason;
    int affectedCell;
    float faultValue;
    bool relayOn, buzzerOn;
    bool unsafeSeen;
    unsigned long unsafeStart;
    unsigned long relayLastOff;
    bool recoverySeen;
    unsigned long recoveryStart;
    int rapidCount[CELL_COUNT];
    bool justTripped, justRecovered;
};

BatteryState battery;
ProtectionKernel protection;

float filteredADC[CELL_COUNT] = {0, 0, 0, 0};
bool firstRead[CELL_COUNT] = {false, false, false, false};

// =====
// SCHEDULER TIMERS
// =====
unsigned long lastADC = 0, lastAnalytics = 0, lastProtection = 0;
unsigned long lastLCD = 0, lastLCDRotate = 0, lastSerial = 0;
uint8_t lcdScreen = 0;

// =====
// BLYNK EVENT-DRIVEN CHANGE-DETECTION CACHE
// =====
BatteryHealth lastBlynkHealth = HEALTHY;
ProtectionState lastBlynkProtection = PROTECTION_SAFE;
int lastBlynkWeakest = -1, lastBlynkStrongest = -1;
float lastBlynkCellVoltage[CELL_COUNT] = {-100, -100, -100, -100};
float lastBlynkPackVoltage = -100, lastBlynkAverageVoltage = -100,
lastBlynkImbalance = -100;
bool lastBlynkRelay = false, lastBlynkBuzzer = false;
bool forceBlynkUpdate = true;

// =====
// TEXT HELPERS
// =====
const char* healthText(BatteryHealth h) {
    switch (h) {
        case HEALTHY: return "HEALTHY";
        case MINOR_IMBALANCE: return "MINOR IMBALANCE";
    }
}

```

```

    case CRITICAL_IMBALANCE: return "CRITICAL";
    case PACK_FAILURE:      return "PACK FAILURE";
    default:                return "UNKNOWN";
}
}

const char* protectionText(ProtectionState s) {
    switch (s) {
        case PROTECTION_SAFE:      return "SAFE";
        case PROTECTION_WARNING:    return "WARNING";
        case PROTECTION_TRIPPED:    return "TRIPPED";
        case PROTECTION_RECOVERY:   return "RECOVERY";
        default:                    return "UNKNOWN";
    }
}

const char* reasonText(ProtectionReason r) {
    switch (r) {
        case REASON_NONE:           return "NONE";
        case REASON_UNDERVOLTAGE:    return "UNDERVOLT";
        case REASON_OVERVOLTAGE:     return "OVERVOLT";
        case REASON_IMBALANCE:       return "IMBALANCE";
        case REASON_RAPID_CHANGE:    return "RAPID CHG";
        case REASON_INVALID_SENSOR:  return "BAD SENSOR";
        default:                     return "UNKNOWN";
    }
}

int healthEnumValue(BatteryHealth h)      { return (int)h; }
int protectionEnumValue(ProtectionState s) { return (int)s; }
int cellEnumValue(int cellIndex)          { return constrain(cellIndex, 0, 3); }

// =====
// ADC READING + SIMULATION MAPPING
// =====
int readAveragedADC(uint8_t pin) {
    long total = 0;
    for (int i = 0; i < ADC_SAMPLE_COUNT; i++) total += analogRead(pin);
    return total / ADC_SAMPLE_COUNT;
}

float adcToCellVoltage(float adc) {
    float ratio = constrain(adc / ADC_MAX_VALUE, 0.0f, 1.0f);
    return SIM_CELL_MIN_V + ratio * (SIM_CELL_MAX_V - SIM_CELL_MIN_V);
}

// =====
// BATTERY: INIT / ACQUIRE / ANALYTICS
// =====
void batteryInit() {
    for (int i = 0; i < CELL_COUNT; i++) {

```

```

    pinMode(CELL_ADC_PIN[i], INPUT);
    analogSetPinAttenuation(CELL_ADC_PIN[i], ADC_11db);
    battery.rawADC[i] = 0;
    battery.cellVoltage[i] = SIM_CELL_MIN_V;
    battery.previousCellVoltage[i] = SIM_CELL_MIN_V;
    battery.sensorValid[i] = true;
    filteredADC[i] = 0;
    firstRead[i] = false;
}
battery.packVoltage = 0;
battery.averageVoltage = 0;
battery.highestVoltage = SIM_CELL_MIN_V;
battery.lowestVoltage = SIM_CELL_MIN_V;
battery.voltageDifference = 0;
battery.imbalancePercent = 0;
battery.strongestCellIndex = 0;
battery.weakestCellIndex = 0;
battery.health = HEALTHY;
}

void batteryAcquire() {
    for (int i = 0; i < CELL_COUNT; i++) {
        int sample = readAveragedADC(CELL_ADC_PIN[i]);
        battery.sensorValid[i] = sample > SENSOR_INVALID_LOW && sample <
SENSOR_INVALID_HIGH;

        if (!firstRead[i]) {
            filteredADC[i] = sample;
            firstRead[i] = true;
        } else {
            filteredADC[i] = FILTER_ALPHA * sample + (1.0f - FILTER_ALPHA) * filteredADC[i];
        }

        battery.rawADC[i] = (int)filteredADC[i];
        battery.previousCellVoltage[i] = battery.cellVoltage[i];
        battery.cellVoltage[i] = adcToCellVoltage(filteredADC[i]);
    }
}

void batteryAnalytics() {
    float sum = 0, highest = battery.cellVoltage[0], lowest = battery.cellVoltage[0];
    int highIndex = 0, lowIndex = 0;
    bool invalidSensor = false;

    for (int i = 0; i < CELL_COUNT; i++) {
        float v = battery.cellVoltage[i];
        sum += v;
        if (v > highest) { highest = v; highIndex = i; }
        if (v < lowest) { lowest = v; lowIndex = i; }
        if (!battery.sensorValid[i]) invalidSensor = true;
    }
}

```

```

battery.packVoltage = sum;
battery.averageVoltage = sum / CELL_COUNT;
battery.highestVoltage = highest;
battery.lowestVoltage = lowest;
battery.strongestCellIndex = highIndex;
battery.weakestCellIndex = lowIndex;
battery.voltageDifference = highest - lowest;
battery.imbalancePercent = (battery.averageVoltage > 0.001f)
    ? (battery.voltageDifference / battery.averageVoltage) * 100.0f : 0;

if (invalidSensor) {
    battery.health = PACK_FAILURE;
} else if (battery.lowestVoltage < MIN_SAFE_VOLTAGE ||
    battery.highestVoltage > MAX_SAFE_VOLTAGE ||
    battery.imbalancePercent > CRITICAL_IMBALANCE_PCT) {
    battery.health = CRITICAL_IMBALANCE;
} else if (battery.imbalancePercent > MINOR_IMBALANCE_PCT) {
    battery.health = MINOR_IMBALANCE;
} else {
    battery.health = HEALTHY;
}
}

// =====
// PROTECTION: UNSAFE-CONDITION DETECTION
// =====
bool findUnsafeCondition(ProtectionReason &reason, int &cell, float &value) {
    for (int i = 0; i < CELL_COUNT; i++) {
        if (!battery.sensorValid[i]) {
            reason = REASON_INVALID_SENSOR; cell = i; value = battery.rawADC[i];
            return true;
        }
    }
    for (int i = 0; i < CELL_COUNT; i++) {
        if (battery.cellVoltage[i] < UNDERVOLTAGE_V) {
            reason = REASON_UNDERVOLTAGE; cell = i; value = battery.cellVoltage[i];
            return true;
        }
    }
    for (int i = 0; i < CELL_COUNT; i++) {
        if (battery.cellVoltage[i] > OVERVOLTAGE_V) {
            reason = REASON_OVERVOLTAGE; cell = i; value = battery.cellVoltage[i];
            return true;
        }
    }
    if (battery.imbalancePercent > IMBALANCE_TRIP_PCT) {
        reason = REASON_IMBALANCE; cell = -1; value = battery.imbalancePercent;
        return true;
    }
    for (int i = 0; i < CELL_COUNT; i++) {

```

```

    float delta = fabsf(battery.cellVoltage[i] - battery.previousCellVoltage[i]);
    protection.rapidCount[i] = (delta > RAPID_MIN_DELTA_V) ? protection.rapidCount[i]
+ 1 : 0;
    if (protection.rapidCount[i] >= RAPID_CONFIRM_COUNT && delta > RAPID_DELTA_V) {
        reason = REASON_RAPID_CHANGE; cell = i; value = delta;
        return true;
    }
}
reason = REASON_NONE; cell = -1; value = 0;
return false;
}

bool recoverySafe() {
    for (int i = 0; i < CELL_COUNT; i++) {
        if (!battery.sensorValid[i]) return false;
        if (battery.cellVoltage[i] < RECOVERY_LOW_V) return false;
        if (battery.cellVoltage[i] > RECOVERY_HIGH_V) return false;
    }
    return battery.imbalancePercent <= MINOR_IMBALANCE_PCT;
}

// =====
// PROTECTION: INIT / STATE MACHINE / OUTPUTS
// =====
void protectionInit() {
    protection.state = PROTECTION_SAFE;
    protection.reason = REASON_NONE;
    protection.affectedCell = -1;
    protection.faultValue = 0;
    protection.relayOn = true;
    protection.buzzerOn = false;
    protection.unsafeSeen = false;
    protection.unsafeStart = 0;
    protection.relayLastOff = 0;
    protection.recoverySeen = false;
    protection.recoveryStart = 0;
    protection.justTripped = false;
    protection.justRecovered = false;
    for (int i = 0; i < CELL_COUNT; i++) protection.rapidCount[i] = 0;

    pinMode(RELAY_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(LED_GREEN_PIN, OUTPUT);
    pinMode(LED_YELLOW_PIN, OUTPUT);
    pinMode(LED_RED_PIN, OUTPUT);

    digitalWrite(RELAY_PIN, HIGH);
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(LED_GREEN_PIN, HIGH);
    digitalWrite(LED_YELLOW_PIN, LOW);
    digitalWrite(LED_RED_PIN, LOW);

```

```
}
```

```
void protectionUpdate() {
    unsigned long now = millis();
    protection.justTripped = false;
    protection.justRecovered = false;

    ProtectionReason reason; int cell; float value;
    bool unsafe = findUnsafeCondition(reason, cell, value);

    switch (protection.state) {
        case PROTECTION_SAFE:
            if (unsafe) {
                protection.state = PROTECTION_WARNING;
                protection.reason = reason; protection.affectedCell = cell;
                protection.faultValue = value;
                protection.unsafeSeen = true; protection.unsafeStart = now;
            }
            break;

        case PROTECTION_WARNING:
            if (!unsafe) {
                protection.state = PROTECTION_SAFE;
                protection.reason = REASON_NONE; protection.affectedCell = -1;
                protection.faultValue = 0;
                protection.unsafeSeen = false;
            } else {
                protection.reason = reason; protection.affectedCell = cell;
                protection.faultValue = value;
                if (now - protection.unsafeStart >= FAULT_CONFIRM_MS) {
                    protection.state = PROTECTION_TRIPPED;
                    protection.relayOn = false; protection.buzzerOn = true;
                    protection.relayLastOff = now; protection.recoverySeen = false;
                    protection.justTripped = true;
                }
            }
            break;

        case PROTECTION_TRIPPED:
            protection.relayOn = false; protection.buzzerOn = true;
            if (now - protection.relayLastOff >= RELAY_MIN_OFF_MS) {
                if (recoverySafe()) {
                    if (!protection.recoverySeen) { protection.recoverySeen = true;
                    protection.recoveryStart = now; }
                    if (now - protection.recoveryStart >= RECOVERY_CONFIRM_MS) protection.state =
                    PROTECTION_RECOVERY;
                } else {
                    protection.recoverySeen = false;
                }
            }
            break;
    }
```

```

case PROTECTION_RECOVERY:
  if (!recoverySafe()) {
    protection.state = PROTECTION_TRIPPED;
    protection.relayOn = false; protection.buzzerOn = true;
    protection.relayLastOff = now; protection.recoverySeen = false;
  } else {
    protection.state = PROTECTION_SAFE;
    protection.relayOn = true; protection.buzzerOn = false;
    protection.reason = REASON_NONE; protection.affectedCell = -1;
  }
  protection.faultValue = 0;
  protection.unsafeSeen = false; protection.recoverySeen = false;
  protection.justRecovered = true;
}
break;
}

// Output policy: relay/buzzer follow the final resolved state
bool safeState = (protection.state == PROTECTION_SAFE ||
  protection.state == PROTECTION_WARNING ||
  protection.state == PROTECTION_RECOVERY);
protection.relayOn = safeState;
protection.buzzerOn = !safeState;
}

void protectionOutputs() {
  digitalWrite(RELAY_PIN, protection.relayOn ? HIGH : LOW);
  digitalWrite(BUZZER_PIN, protection.buzzerOn ? HIGH : LOW);
  digitalWrite(LED_GREEN_PIN, protection.state == PROTECTION_SAFE ? HIGH : LOW);
  digitalWrite(LED_YELLOW_PIN, protection.state == PROTECTION_WARNING ? HIGH : LOW);
  digitalWrite(LED_RED_PIN, (protection.state == PROTECTION_TRIPPED ||
    protection.state == PROTECTION_RECOVERY) ? HIGH : LOW);
}

// =====
// BLYNK: SEND FULL SNAPSHOT (matches your V0-V13 exactly)
// =====
void sendAllBlynkData() {
  if (!Blynk.connected()) return;

  Blynk.virtualWrite(BLYNK_RELAY, protection.relayOn ? 1 : 0);
  Blynk.virtualWrite(BLYNK_BUZZER, protection.buzzerOn ? 1 : 0);

  Blynk.virtualWrite(BLYNK_CELL1, battery.cellVoltage[0]);
  Blynk.virtualWrite(BLYNK_CELL2, battery.cellVoltage[1]);
  Blynk.virtualWrite(BLYNK_CELL3, battery.cellVoltage[2]);
  Blynk.virtualWrite(BLYNK_CELL4, battery.cellVoltage[3]);

  Blynk.virtualWrite(BLYNK_PACK_VOLTAGE, battery.packVoltage);
  Blynk.virtualWrite(BLYNK_AVERAGE_VOLTAGE, battery.averageVoltage);
  Blynk.virtualWrite(BLYNK_IMBALANCE, battery.imbalancePercent);
}

```

```

    Blynk.virtualWrite(BLYNK_WEAKEST_CELL, cellEnumValue(battery.weakestCellIndex));
    Blynk.virtualWrite(BLYNK_STRONGEST_CELL,
cellEnumValue(battery.strongestCellIndex));

    Blynk.virtualWrite(BLYNK_SYSTEM_STATUS, protectionEnumValue(protection.state));
    Blynk.virtualWrite(BLYNK_HEALTH, healthEnumValue(battery.health));
    Blynk.virtualWrite(BLYNK_PROTECTION, protectionEnumValue(protection.state));
}
// =====
// BLYNK: EVENT-DRIVEN CHANGE DETECTION (sends only on real change)
// =====
void checkBlynkEvents() {
    bool eventDetected = false;

    for (int i = 0; i < CELL_COUNT; i++) {
        if (fabsf(battery.cellVoltage[i] - lastBlynkCellVoltage[i]) >= 0.02f) {
            eventDetected = true;
            lastBlynkCellVoltage[i] = battery.cellVoltage[i];
        }
    }
    if (fabsf(battery.packVoltage - lastBlynkPackVoltage) >= 0.05f) {
        eventDetected = true; lastBlynkPackVoltage = battery.packVoltage;
    }
    if (fabsf(battery.averageVoltage - lastBlynkAverageVoltage) >= 0.02f) {
        eventDetected = true; lastBlynkAverageVoltage = battery.averageVoltage;
    }
    if (fabsf(battery.imbalancePercent - lastBlynkImbalance) >= 0.20f) {
        eventDetected = true; lastBlynkImbalance = battery.imbalancePercent;
    }
    if (battery.health != lastBlynkHealth) { eventDetected = true; lastBlynkHealth =
battery.health; }
    if (protection.state != lastBlynkProtection) { eventDetected = true;
lastBlynkProtection = protection.state; }
    if (battery.weakestCellIndex != lastBlynkWeakest) { eventDetected = true;
lastBlynkWeakest = battery.weakestCellIndex; }
    if (battery.strongestCellIndex != lastBlynkStrongest) { eventDetected = true;
lastBlynkStrongest = battery.strongestCellIndex; }
    if (protection.relayOn != lastBlynkRelay) { eventDetected = true; lastBlynkRelay =
protection.relayOn; }
    if (protection.buzzerOn != lastBlynkBuzzer) { eventDetected = true;
lastBlynkBuzzer = protection.buzzerOn; }

    if (eventDetected || forceBlynkUpdate) {
        sendAllBlynkData();
        forceBlynkUpdate = false;
    }
}

BLYNK_CONNECTED() {
    Serial.println(F("\n====="));
    Serial.println(F("BLYNK CLOUD CONNECTED"));
}

```

```

Serial.println(F("====="));
forceBlynkUpdate = true;
}
// =====
// LCD (4 rotating screens, cleared once per rotation)
// =====
void showLCD() {
  unsigned long now = millis();
  if (now - lastLCDRotate >= LCD_ROTATE_MS) {
    lastLCDRotate = now;
    lcdScreen = (lcdScreen + 1) % 4;
    lcd.clear();
  }
  switch (lcdScreen) {
    case 0:
      lcd.setCursor(0, 0); lcd.print("4-CELL BMS MONITOR");
      lcd.setCursor(0, 1); lcd.print("C1:"); lcd.print(battery.cellVoltage[0], 2);
      lcd.print(" C2:"); lcd.print(battery.cellVoltage[1], 2);
      lcd.setCursor(0, 2); lcd.print("C3:"); lcd.print(battery.cellVoltage[2], 2);
      lcd.print(" C4:"); lcd.print(battery.cellVoltage[3], 2);
      lcd.setCursor(0, 3); lcd.print("Health:");
      lcd.print(healthText(battery.health));
      break;

    case 1:
      lcd.setCursor(0, 0); lcd.print("PACK ANALYTICS");
      lcd.setCursor(0, 1); lcd.print("Pack :"); lcd.print(battery.packVoltage, 2);
      lcd.print("V");
      lcd.setCursor(0, 2); lcd.print("Avg  :"); lcd.print(battery.averageVoltage, 2);
      lcd.print("V");
      lcd.setCursor(0, 3); lcd.print("Imbal:"); lcd.print(battery.imbalancePercent,
1); lcd.print("%");
      break;

    case 2:
      lcd.setCursor(0, 0); lcd.print("CELL ANALYSIS");
      lcd.setCursor(0, 1); lcd.print("Strong: C");
      lcd.print(battery strongestCellIndex + 1);
      lcd.print(" "); lcd.print(battery.highestVoltage, 2); lcd.print("V");
      lcd.setCursor(0, 2); lcd.print("Weak  : C"); lcd.print(battery.weakestCellIndex
+ 1);
      lcd.print(" "); lcd.print(battery.lowestVoltage, 2); lcd.print("V");
      lcd.setCursor(0, 3); lcd.print("Diff:"); lcd.print(battery.voltageDifference,
2); lcd.print("V");
      break;

    case 3:
      lcd.setCursor(0, 0); lcd.print("PROTECTION");
      lcd.setCursor(0, 1); lcd.print("State:");
      lcd.print(protectionText(protection.state));

```

```

    lcd.setCursor(0, 2); lcd.print("Relay:"); lcd.print(protection.relayOn ? "ON " :
"OFF");
    lcd.setCursor(10, 2); lcd.print("Buzz:"); lcd.print(protection.buzzerOn ? "ON" :
"OFF");
    lcd.setCursor(0, 3); lcd.print("Rsn:");
    lcd.print(reasonText(protection.reason));
    break;
}
}
// =====
// SERIAL REPORT
// =====
void printSerialReport() {
    Serial.println();
    Serial.println(F("====="));
    Serial.println(F("          4-CELL BMS SYSTEM STATUS"));
    Serial.println(F("====="));

    for (int i = 0; i < CELL_COUNT; i++) {
        Serial.print("Cell "); Serial.print(i + 1); Serial.print(" : ");
        Serial.print(battery.cellVoltage[i], 3); Serial.print(" V | ADC=");
        Serial.print(battery.rawADC[i]); Serial.print(" | Sensor=");
        Serial.println(battery.sensorValid[i] ? "OK" : "INVALID");
    }

    Serial.println(F("-----"));
    Serial.print(F("Pack Voltage : ")); Serial.print(battery.packVoltage, 3);
Serial.println(" V");
    Serial.print(F("Average      : ")); Serial.print(battery.averageVoltage, 3);
Serial.println(" V");
    Serial.print(F("Highest      : ")); Serial.print(battery.highestVoltage, 3);
Serial.println(" V");
    Serial.print(F("Lowest      : ")); Serial.print(battery.lowestVoltage, 3);
Serial.println(" V");
    Serial.print(F("Difference  : ")); Serial.print(battery.voltageDifference, 3);
Serial.println(" V");
    Serial.print(F("Imbalance   : ")); Serial.print(battery.imbalancePercent, 2);
Serial.println(" %");
    Serial.print(F("Strongest   : Cell ")); Serial.println(battery.strongestCellIndex
+ 1);
    Serial.print(F("Weakest     : Cell ")); Serial.println(battery.weakestCellIndex +
1);
    Serial.print(F("Health      : ")); Serial.println(healthText(battery.health));

    Serial.println(F("-----"));
    Serial.print(F("Protection   : "));
Serial.println(protectionText(protection.state));
    Serial.print(F("Reason      : ")); Serial.println(reasonText(protection.reason));
    Serial.print(F("Relay       : ")); Serial.println(protection.relayOn ? "ON" :
"OFF");

```

```

Serial.print(F("Buzzer      : ")); Serial.println(protection.buzzerOn ? "ON" :
"OFF");

if (protection.affectedCell >= 0) {
  Serial.print(F("Fault Cell   : Cell ")); Serial.println(protection.affectedCell +
1);
  Serial.print(F("Fault Value   : ")); Serial.println(protection.faultValue, 3);
}
Serial.println(F("====="));
}

// =====
// SETUP
// =====
void setup() {
  Serial.begin(115200);

  Wire.begin(LCD_SDA_PIN, LCD_SCL_PIN);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("INTELLIGENT BMS");
  lcd.setCursor(0, 1); lcd.print("ESP32 STARTING...");
  lcd.setCursor(0, 2); lcd.print("4 CELL SIMULATION");
  lcd.setCursor(0, 3); lcd.print("BLYNK ENABLED");

  batteryInit();
  protectionInit();
  batteryAcquire();
  batteryAnalytics();
  protectionUpdate();
  protectionOutputs();

  Serial.println(F("\nConnecting to Blynk..."));
  WiFi.mode(WIFI_STA);
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

  blynkTimer.setInterval(250L, checkBlynkEvents);
  blynkTimer.setInterval(BLYNK_HEARTBEAT_MS, []() {
    if (Blynk.connected()) sendAllBlynkData();
  });

  unsigned long now = millis();
  lastADC = now; lastAnalytics = now; lastProtection = now;
  lastLCD = now; lastLCDRotate = now; lastSerial = now;

  forceBlynkUpdate = true;

  Serial.println(F("\n====="));
  Serial.println(F("BMS SYSTEM BOOT COMPLETE"));
  Serial.println(F("BLYNK V0-V13 ENABLED"));

```

```

    Serial.println(F("====="));
}
// =====
// MAIN LOOP - fully non-blocking, millis()-scheduled
// =====
void loop() {
    unsigned long now = millis();

    Blynk.run();
    blynkTimer.run();

    if (now - lastADC >= ADC_INTERVAL_MS) {
        lastADC = now;
        batteryAcquire();
    }

    if (now - lastAnalytics >= ANALYTICS_INTERVAL_MS) {
        lastAnalytics = now;
        batteryAnalytics();
    }

    if (now - lastProtection >= PROTECTION_INTERVAL_MS) {
        lastProtection = now;
        protectionUpdate();
        protectionOutputs();

        if (protection.justTripped) {
            Serial.println(F("\n*** SAFETY TRIP ***"));
            Serial.print(F("Reason: ")); Serial.println(reasonText(protection.reason));
            forceBlynkUpdate = true;
        }
        if (protection.justRecovered) {
            Serial.println(F("\n*** SYSTEM RECOVERED ***"));
            forceBlynkUpdate = true;
        }
    }

    if (now - lastLCD >= LCD_INTERVAL_MS) {
        lastLCD = now;
        showLCD();
    }

    if (now - lastSerial >= SERIAL_INTERVAL_MS) {
        lastSerial = now;
        printSerialReport();
    }
}

```

11. Wokwi Simulation

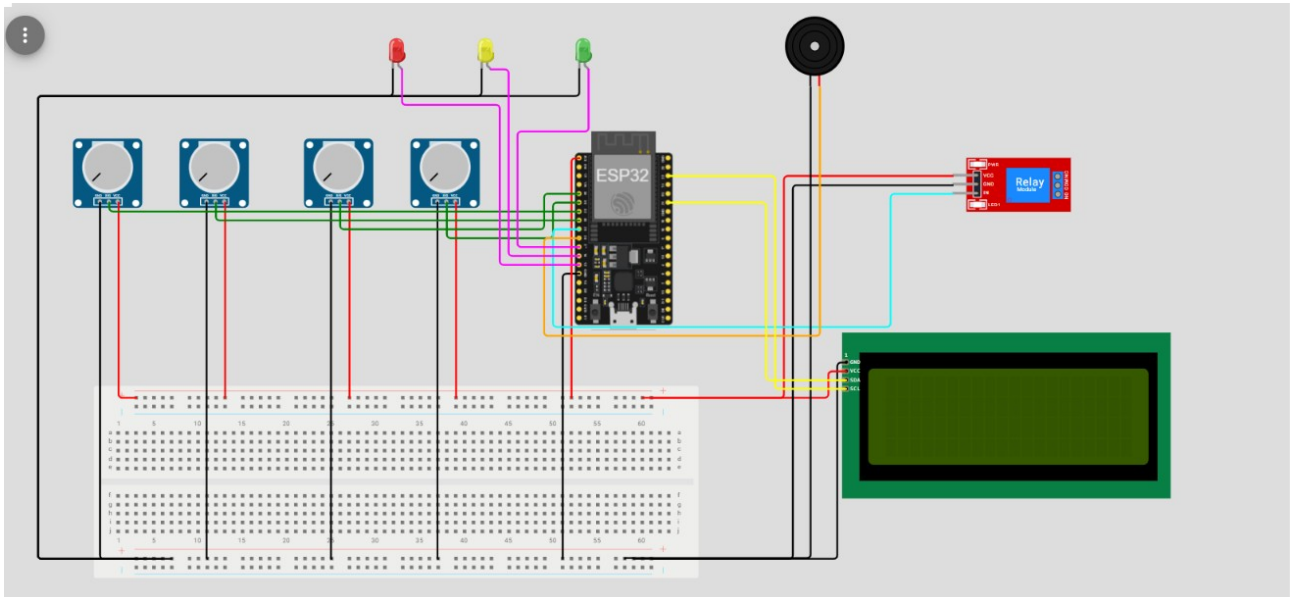
Wokwi is used to safely simulate the complete system.

The simulation demonstrates:

- Normal operation
 - Cell imbalance
 - Undervoltage
 - Overvoltage
 - Sensor faults
 - Rapid voltage changes
 - Protection trips
 - Recovery
 - Wi-Fi disconnection/reconnection
- Four potentiometers are used as simulated cell-voltage inputs.

Wokwi link: <https://wokwi.com/projects/476036431640472577>

Circuit diagram:



12. Blynk Cloud Monitoring

Blynk provides remote monitoring through Wi-Fi.

The dashboard displays:

- System status
- Cell voltages
- Battery analytics
- Historical trends
- Protection status
- Diagnostics
- Recommendations

An important design feature is that **local battery protection continues independently of cloud connectivity**.

Blynk web dashboard:



13. Verification and Results

The system is verified by checking:

- Cell measurements
- Battery calculations
- Health classification
- Fault detection
- Protection state
- Relay operation
- Buzzer and LED status
- LCD information
- Blynk telemetry
- Recovery behavior

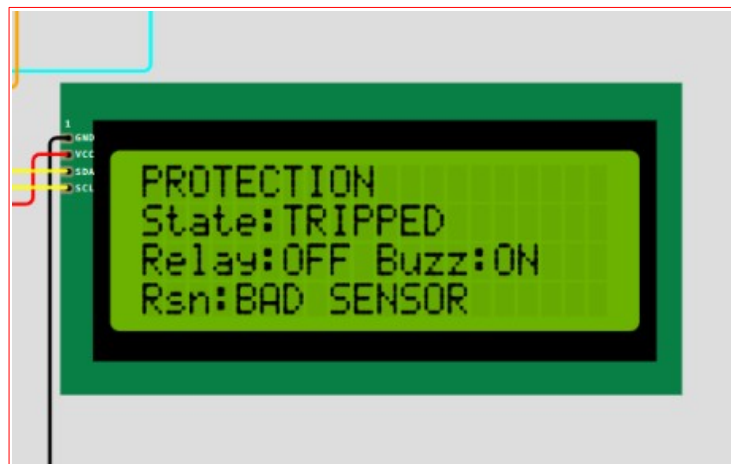
The project documentation defines **15 test cases** covering the major operating and fault conditions.

OUTPUT:

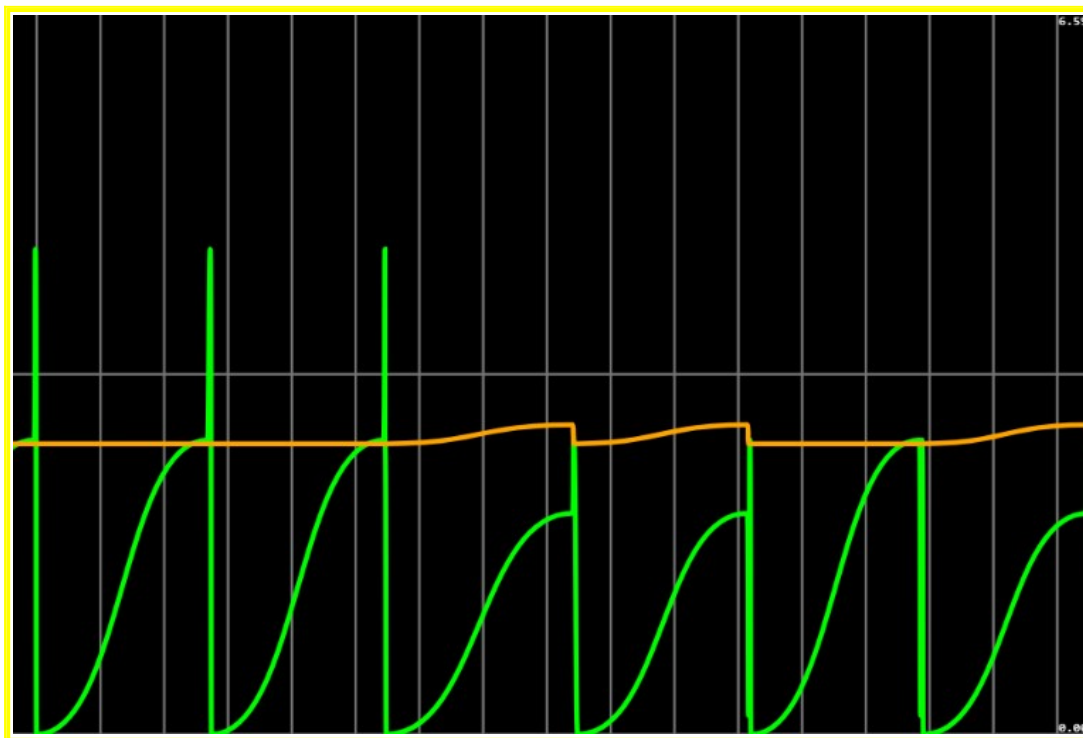
```
===== 4-CELL BMS =====  
Cell 1: 2.900 V  
Cell 2: 2.852 V  
Cell 3: 4.200 V  
Cell 4: 2.890 V  
Pack: 12.842 V  
Average: 3.211 V  
Imbalance: 41.98 %  
Health: CRITICAL  
Protection: TRIPPED  
Reason: CRITICAL IMBALANCE  
Relay: OFF  
=====
```

```
===== 4-CELL BMS =====  
Cell 1: 4.200 V  
Cell 2: 2.500 V  
Cell 3: 4.200 V  
Cell 4: 4.200 V  
Pack: 15.100 V  
Average: 3.775 V  
Imbalance: 45.03 %  
Health: CRITICAL  
Protection: TRIPPED  
Reason: UNDERVOLTAGE  
Relay: OFF  
=====
```

LCD interface output:



Waveform:



14. Limitations

The current system is a **simulation-based prototype**, not a production-certified BMS.

Main limitations:

- Potentiometers do not represent real battery chemistry.
- No battery current measurement.
- No physical cell balancing.

- No real temperature monitoring.
 - No certified battery safety thresholds.
 - Wokwi cannot fully reproduce real battery electrical and thermal behavior.
-

15. Future Enhancements

Future versions can include:

- Current sensor
 - State-of-Charge estimation
 - Temperature monitoring
 - Active/passive cell balancing
 - Persistent fault storage
 - OTA firmware updates
 - MQTT communication
 - AI-based battery fault prediction and degradation analysis
-

16. Conclusion

The **Intelligent 4-Cell Battery Management System using ESP32** successfully combines battery monitoring, analytics, safety protection, fault detection, HMI, and cloud monitoring in one embedded system.

The system continuously monitors four battery cells and calculates important parameters such as **cell voltage, pack voltage, average voltage, voltage difference, and imbalance percentage**. The event-driven safety system detects abnormal conditions and manages them through **SAFE, WARNING, TRIPPED, and RECOVERY** states.

The **fault-tolerant runtime** helps identify sensor and system failures, while the **LCD HMI** provides clear battery and diagnostic information. Blynk cloud telemetry enables remote monitoring without affecting the local safety functions. The complete system uses a **non-blocking millis()-based architecture**, allowing different functions to work together efficiently.

Overall, this project demonstrates how an ESP32 can be used to build a structured, reliable, and intelligent battery monitoring and protection system. The simulation also provides a strong foundation for future development of real-world battery management systems.

“A battery may store energy, but intelligent engineering gives that energy direction, protection, and purpose.”